```
# Task 1 - String Operations

text = "DataScienceWithPython"

print("Extract Data:", text[:4])
print("Extract Python:", text[-6:])
print("Reverse String:", text[::-1])
print("Every 2nd Character:", text[::2])
print("Lowercase:", text.lower())

vowels = "aeiouAEIOU"
count = 0
for ch in text:
    if ch in vowels:
        count += 1
print("Number of Vowels:", count)

# Task 2 - List Operations
numbers = [10, 20, 30, 40, 50, 20, 10]

numbers.append(60)
numbers.insert(1, 15)

numbers = list(dict.fromkeys(numbers))   # Remove duplicates

numbers.sort(reverse=True)

print("List after operations:", numbers)
print("Frequency of 20:", numbers.count(20))

tuple_numbers = tuple(numbers)
print("Converted to Tuple:", tuple_numbers)
```

```
# Task 3 - Tuple Operations
data = (5, 10, 15, 20, 25, 10)

print("Display 20:", 20 in data)

temp_list = list(data)
temp_list.append(30)
print("After append:", temp_list)

print("Slice index 1 to 4:", data[1:5])

# Task 4 - Dictionary Operations
student = {
    "name": "Rahul",
    "age": 21,
    "marks": 85
}

student["grade"] = "A"
student["marks"] = 90
student.pop("age")

print("Keys:", student.keys())
print("Values:", student.values())
print("Is 'name' key exists?", "name" in student)

# Task 5 - Set Operations
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

print("Union:", A | B)
print("Intersection:", A & B)
```

```
print("Difference (A-B):", A - B)
print("Symmetric Difference:", A ^ B)

A.add(10)
A.remove(2)
print(A)
```

**# Task 6 - Functions**

```
# Prime check
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

# Factorial using recursion
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)

# Fibonacci series
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=" ")
        a, b = b, a+b
    print()
```

```
# Default argument
def greet(name="Student"):
    print("Hello", name)

# Lambda square
square = lambda x: x*x

print("Prime 7:", is_prime(7))
print("Factorial 5:", factorial(5))
print("Fibonacci series:")
fibonacci(6)
greet()
print("Square of 5:", square(5))
```

**# Task 7 - Patterns & Logic**

```
# Multiplication table
num = 5
for i in range(1, 11):
    print(num, "x", i, "=", num*i)

# Star pattern
for i in range(1, 6):
    print("*" * i)

# Sum of digits
number = 12345
total = sum(int(d) for d in str(number))
print("Sum of digits:", total)

# Count even & odd
lst = [1,2,3,4,5,6]
even = sum(1 for x in lst if x%2==0)
```

```
odd = sum(1 for x in lst if x%2!=0)
print("Even:", even, "Odd:", odd)

# First 10 primes
count = 0
num = 2
while count < 10:
    if is_prime(num):
        print(num, end=" ")
        count += 1
    num += 1
print()
```

**# Task 8 - OOP**

```
class Student:
    school = "ABC College"   # Class variable

    def __init__(self, name, age, marks):
        self.name = name      # Instance variable
        self.age = age
        self.__marks = marks  # Private variable (Encapsulation)

    def display(self):
        print(self.name, self.age, self.__marks)

class Rectangle:
    def __init__(self, l, w):
        self.l = l
        self.w = w

    def area(self):
        return self.l * self.w
```

```
    def perimeter(self):
        return 2*(self.l + self.w)

# Inheritance & Overriding
class Child(Student):
    def display(self):
        print("Overridden Method")

s = Student("Rahul", 21, 85)
s.display()

r = Rectangle(5,4)
print("Area:", r.area())
print("Perimeter:", r.perimeter())
```

**# Task 9 - NumPy**

```
import numpy as np

arr = np.arange(1,11)
print("1D Array:", arr)

zeros = np.zeros((3,3))
print("3x3 Zeros:\n", zeros)

identity = np.eye(3)
print("Identity Matrix:\n", identity)

random_nums = np.random.randint(1,101,5)
print("Random Integers:", random_nums)

reshaped = arr.reshape(2,5)
```

```
print("Reshaped:\n", reshaped)

print("Sorted Desc:", np.sort(arr)[::-1])
```

**# Task 10 - Pandas**

```
import pandas as pd

df = pd.DataFrame({
    "Name":["A","B","C"],
    "Marks":[85,90,None],
    "Age":[20,21,19]
})

print(df.head())
print(df.describe())

df["Marks"].fillna(df["Marks"].mean(), inplace=True)
df.drop_duplicates(inplace=True)
df.rename(columns={"Marks":"Score"}, inplace=True)

df.to_csv("cleaned_data.csv", index=False)
```

**# Task 11 - Matplotlib**

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,30,40]

plt.figure(figsize=(8,6))

plt.subplot(2,1,1)
plt.plot(x,y)
```

```
plt.title("Line Plot")

plt.subplot(2,1,2)
plt.bar(x,y)
plt.title("Bar Chart")

plt.tight_layout()
plt.show()
```

**# Task 12 - Pillow**

```
from PIL import Image, ImageFilter

img = Image.open("image.jpg")
img.show()

img = img.resize((200,200))
img.show()

img = img.convert("L")
img.show()
img = img.rotate(45)
img.show()
img = img.crop((10,10,180,180))
img.show()
img = img.filter(ImageFilter.BLUR)
img.show()

img.save("modified.png")
```

Outputs:

```
(.venv) PS D:\Internship\Day23> python task1-12.py
Extract Data: Data
Extract Python: Python
Reverse String: nohtyPhtiWecneicSataD
Every 2nd Character: DtSineihyhn
Lowercase: datasciencewithpython
Number of Vowels: 7
List after operations: [60, 50, 40, 30, 20, 15, 10]
Frequency of 20: 1
Converted to Tuple: (60, 50, 40, 30, 20, 15, 10)
Display 20: True
After append: [5, 10, 15, 20, 25, 10, 30]
Slice index 1 to 4: (10, 15, 20, 25)
Keys: dict_keys(['name', 'marks', 'grade'])
Values: dict_values(['Rahul', 90, 'A'])
Is 'name' key exists? True
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
{1, 3, 4, 10}
Prime 7: True
Factorial 5: 120
Fibonacci series:
0 1 1 2 3 5
Hello Student
Square of 5: 25
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

```
*
**
***
****
*****
Sum of digits: 15
Even: 3 Odd: 3
2 3 5 7 11 13 17 19 23 29
Rahul 21 85
Area: 20
Perimeter: 18
1D Array: [ 1  2  3  4  5  6  7  8  9 10]
3x3 Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
Identity Matrix:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
Random Integers: [ 1 48 51 33 40]
Reshaped:
 [[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
Sorted Desc: [10  9  8  7  6  5  4  3  2  1]
  Name  Marks  Age
0    A   85.0   20
1    B   90.0   21
2    C    NaN   19
            Marks   Age
count   2.000000   3.0
mean   87.500000  20.0
std     3.535534   1.0
min    85.000000  19.0
25%    86.250000  19.5
50%    87.500000  20.0
75%    88.750000  20.5
max    90.000000  21.0
```

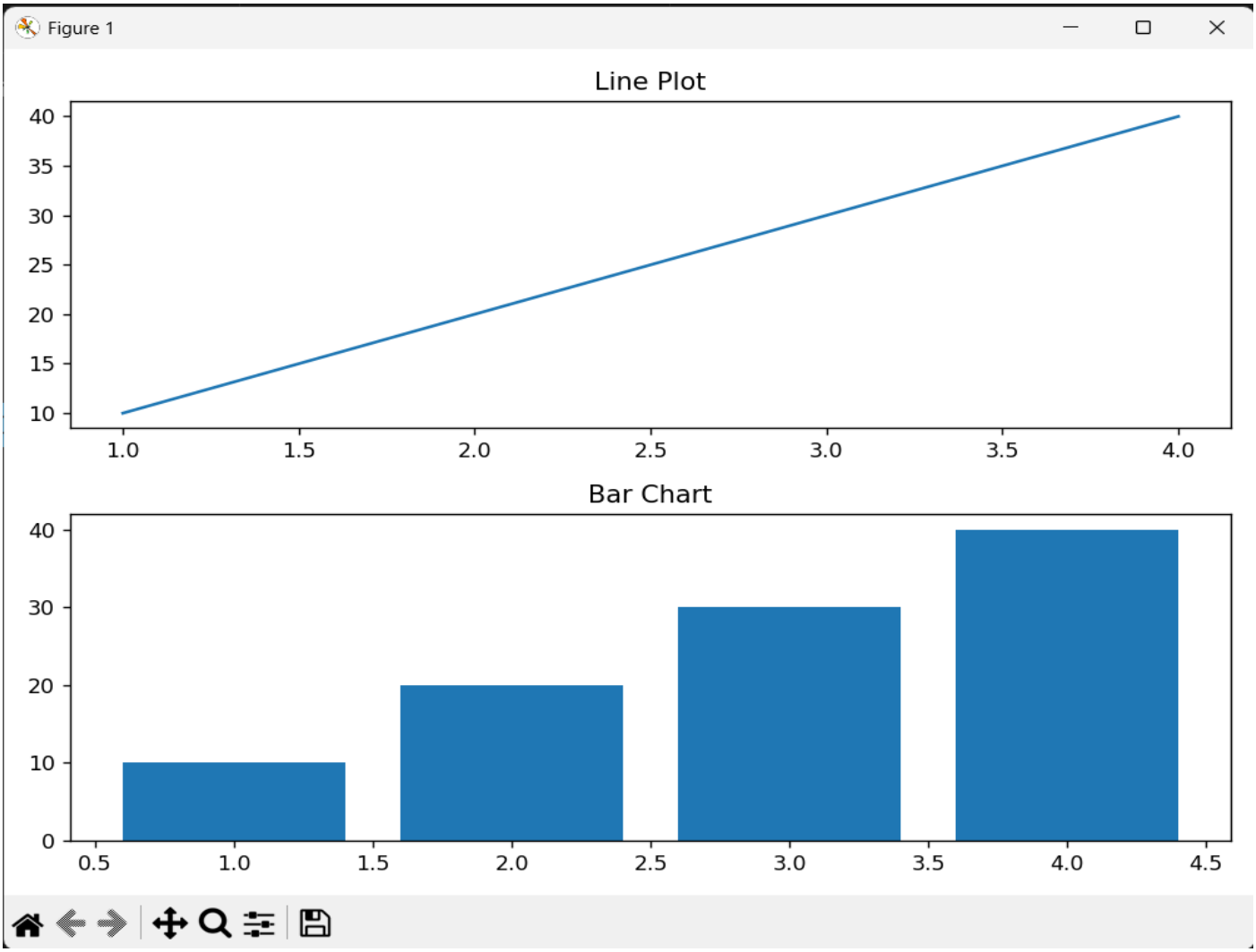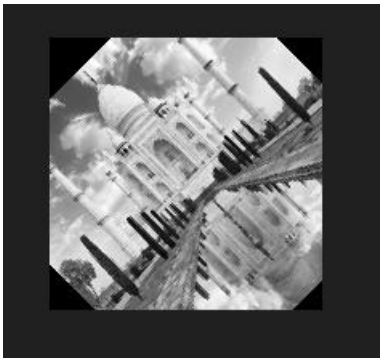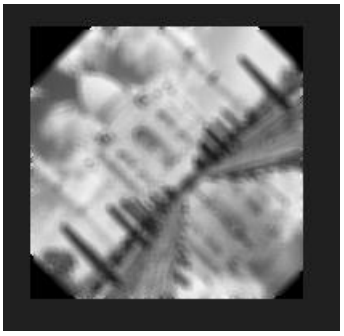
Figure 1
Line Plot
40
35
30
25
20
15
10
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Bar Chart
40
30
20
10
0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5